

AI Deep Drawability

July 2026 Project Update

Date: 2026-07-12

Author: Paul Lintzen

1 Updates

1.1 Sheet Classifier Model

This week I focused on building a new model architecture that produces probabilities for a sample being from each distinct sheet alongside the binary good/bad output. The idea behind this new architecture is that by explicitly forcing the model to predict which sheet an image came from before giving its final binary classification we might be able to improve its understanding of sheet-to-sheet variability versus features that indicate if it is good or bad.

The big limitation here is that we are still limited to 8 sheets (or 6 when using 2 for holdouts) so for unseen sheets the actual sheet classification is meaningless. What we hope is that unseen good sheets will be more similar to seen good sheets and vice versa meaning that when we look at the majority vote for good vs. bad sheets in our model the correct one wins out.

In order to make this generalize our loss term will now have two components. One for getting the final sheet classification correct and one for getting the final good/bad classification correct. Figuring out how to balance these will likely be key to allowing this model to function.

Last week I found that higher dimensional embeddings past 768 don't seem to give any additional performance benefit so I will stick with that size. The best results we saw (though only by a small margin) were with the DinoV3-vitb16 model so I will use that embedding for the model.

1.2 Results

I started with the naive approach of just training one model on all the sheets holding on 15% of samples randomly. Counting both losses equally I ended up with a validation accuracy of 0.7305 for the sheets and 0.9190 for the labels. Weighting either of these losses as 0 gave slightly worse final validation accuracy than the 50/50 weights and as would be expected much lower accuracy for the dropped term. So it seems like considering both the sheet classification and final label is leading to some performance improvement though it is very small.

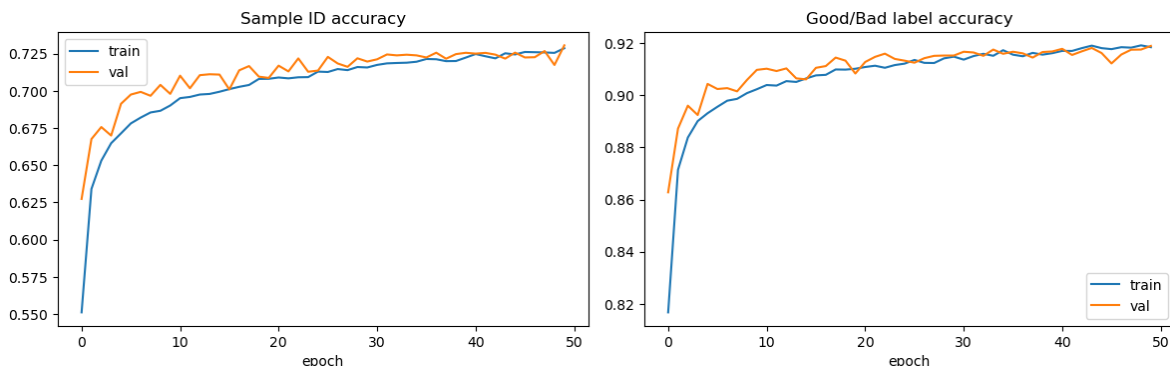


Figure 1: 1:1 weights sheet classifier model

Sample Weight	Label Weight	Sample Val Acc	Sample Label Acc
1	1	0.7305	0.9190
1	0	0.7289	0.3482
0	1	0.1371	0.9127
10	1	0.7257	0.9072
1	10	0.7109	0.9152

Table 1: Loss weights vs validation accuracy

Comparing to the previous model, this 0.92 validation accuracy is within error so there doesn't seem to be a meaningful difference at this point. The real test will be this model's performance on each holdout dataset.

Retraining this new model using the same 15 holdout combinations we got similar results to before. Figure 2 shows the breakdown of accuracy per sheet. Comparing this to the results we were getting with the previous model (in Figure 3) we see a slight increase in accuracy in 11/15 of the holdout models, a decrease in 3, and a match in 1. Overall, it seems that this model is performing slightly better than the previous architecture but not by much.

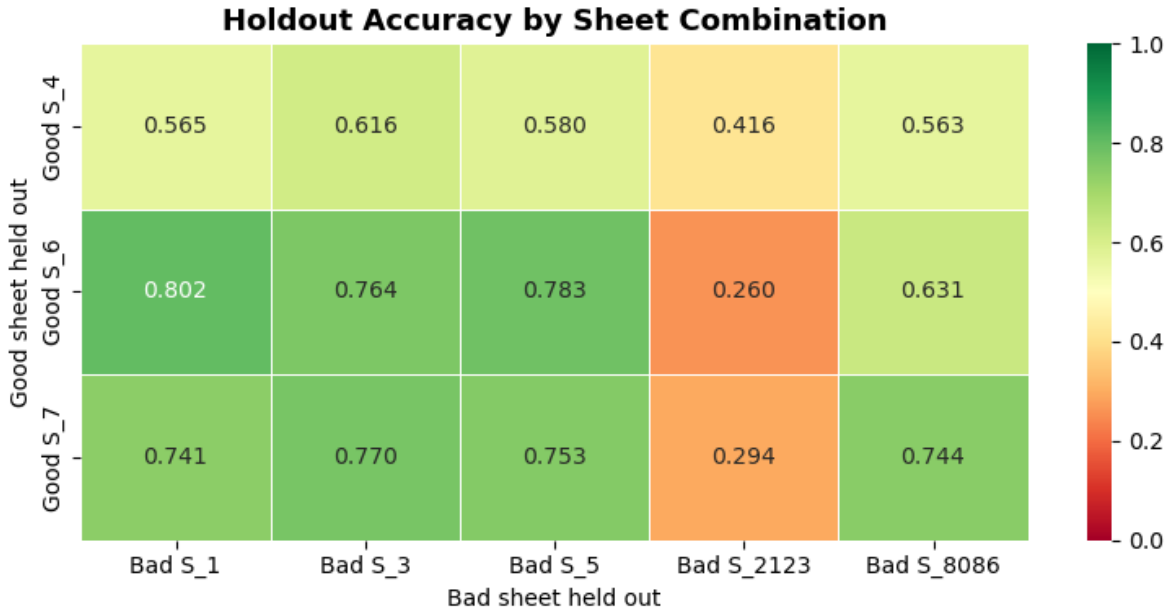


Figure 2: 1:1 model accuracy by holdout

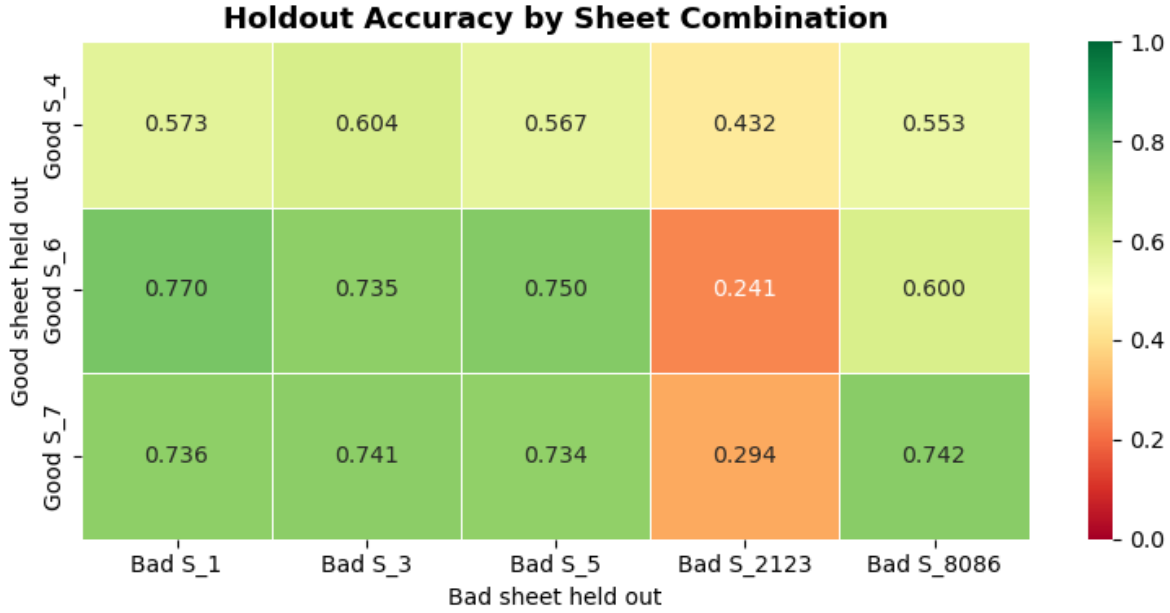


Figure 3: Previous model accuracy break down

Figure 4 shows the average accuracy between good and bad across all sheets. This is more or less on par with the results we had last week. The combined accuracy here is 0.604 compared to the absolute best of 0.611 we saw previously. These differences are so small that they are likely more due to random chance than meaningful improvement.

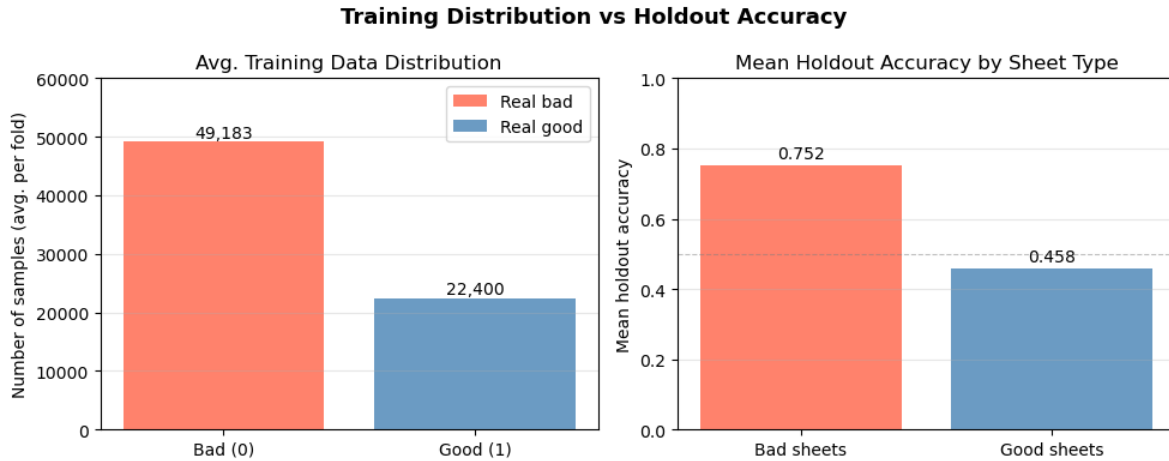


Figure 4: Unweighted model good/bad accuracies

We found last week that class weighting does seem to improve performance slightly at no real performance hit so implementing that for this new architecture we see a further small combined increase. We trade off a small drop in bad sheet accuracy for a larger in good sheet accuracy. Figure 5 shows the results and we see our best overall accuracy yet of 0.612 but being just 0.01 better than our previous best it isn't a meaningful boost.

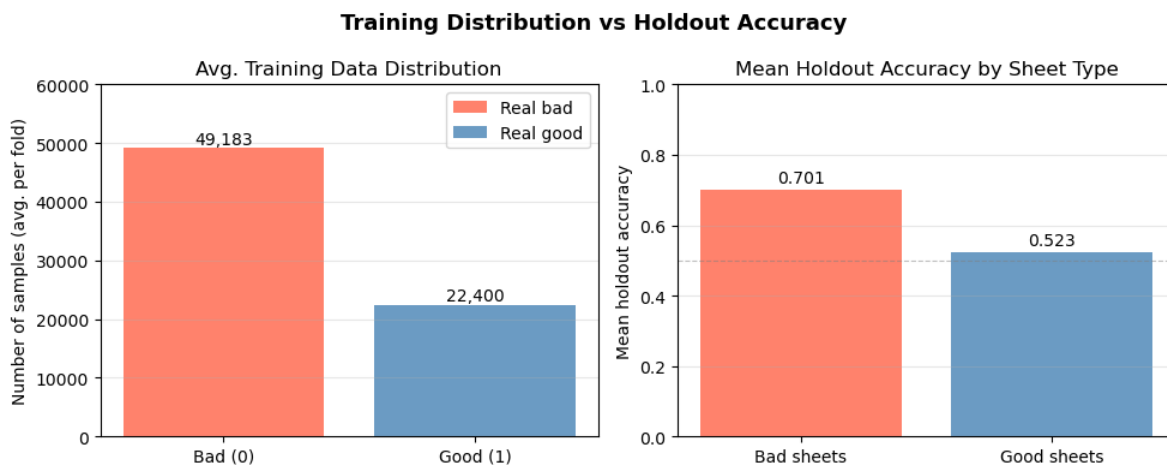


Figure 5: Weighted model good/bad accuracies

1.3 Model Revision

One final change I tested was using the sample predictions directly. Rather than just using both sample and label prediction as two distinct outputs for training the model, I am now only predicting the sample probabilities and storing a map of which samples are good vs. bad. This allows us to aggregate the probabilities for each sample to predict the final good/bad label. The results for this final model are shown in Figure 6.

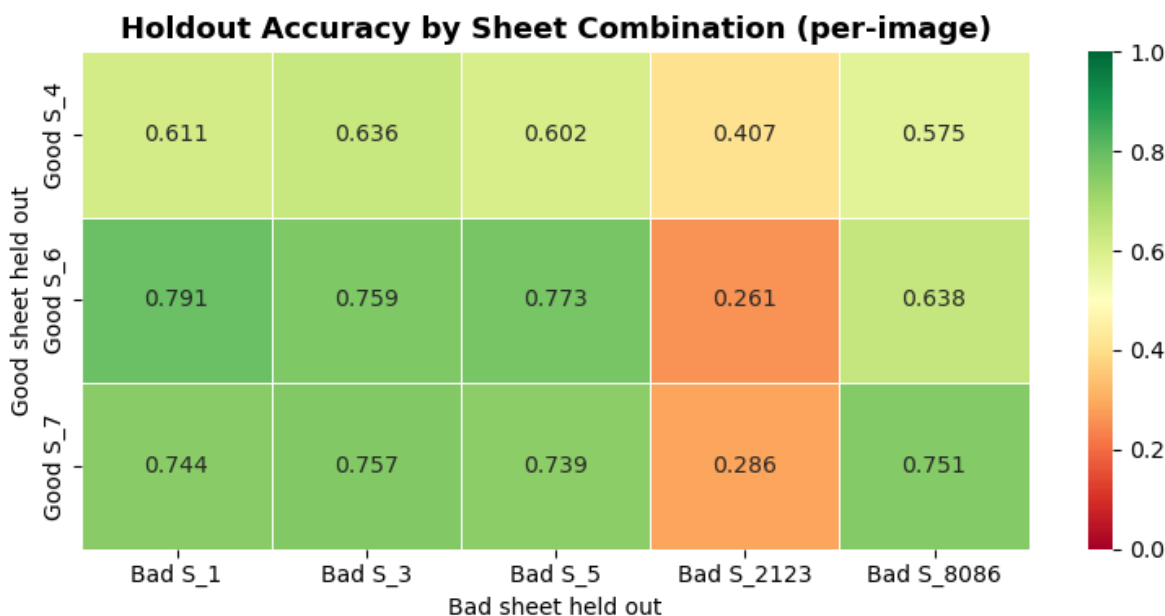


Figure 6: Adjusted model accuracies

This got the bad accuracy up to 0.759 and kept the good at 0.458 making it slightly better than the previous unweighted model. Comparing accuracies for each holdout we see a small boost for most but a couple went down. The overall accuracy is 0.609 so again within the range of what we've been seeing and not clearly better or worse.

1.4 Conclusion

Overall, this was a valuable exploration into a different architecture but again seems to confirm that our current limitation is data. The differences we are seeing from model to model here don't seem to indicate that our previous architecture was bad or lacking but rather that the data is truly difficult to separate and we need more in order to accurately generalize to unseen sheets.

2 Next Steps

Next steps haven't meaningfully changed since last week. SMOTE doesn't seem to be helpful but class weighting makes a small improvement. I still plan to try contrastive learning as another different architecture but I'm not expecting big performance improvements over the current level. I will also consult with Dr. Pak to get his recommendations on next steps as well.